Laboratório de Sistemas Operacionais – sched_profiler

Calvin Silva, João Medeiros, Pedro Jucewicz · PUCRS · novembro de 2026

1. Programa e metodologia

```
sched_profiler <tam_buffer> <num_threads>
               <pol1>[:prio1] [<pol2>[:prio2] ...]
```

Cada thread recebe sua política e prioridade (se faltarem, a última se repete) e escreve sua letra no buffer global sob um semáforo binário; uma barreira (`pthread_barrier_t`, N+1 participantes) garante que todas começam juntas, já com a política aplicada. `SCHED_LOW_IDLE` vale 7, como no kernel. Além do buffer e do resumo, o programa imprime por thread os **escalonamentos** (blocos no buffer), os **caracteres escritos**, o **tempo de CPU** e as **trocas de contexto** voluntárias/involuntárias; as duas últimas vêm do kernel e não dependem do semáforo.

Ambiente. Host: GitHub Codespace de 2 núcleos (AMD EPYC 9V74, 2 vCPUs), 7,8 GiB de RAM, Ubuntu 24.04. Guest: QEMU 8.2.2 (qemu-system-i386, emulação por software, sem KVM), 128 MiB de RAM (padrão), 1 ou 2 vCPUs (`--smp cpus=1|2`), distribuição Buildroot com Linux 4.13.9 e `SCHED_LOW_IDLE` (peso 1, preemptada ao acordar qualquer tarefa de outra política), com `WMULT_LOW_IDLEPRIO` corrigido para $2^{32}/1$ (antes era $2^{32}/3$, o inverso do peso da IDLE). Buffer de 5.000.000 posições; 11 experimentos com 1 núcleo e 5 com 2 núcleos, cada um com `trace-cmd record -e sched:sched_switch` durante a execução.

Nota: o script dos experimentos salva a saída truncando linhas com mais de 100 caracteres (o buffer e os resumos longos) para 60 caracteres e "...". Isso ocorre só no arquivo salvo, após a execução; os valores por thread são calculados pelo programa sobre o buffer completo e não são afetados.

2. Resultados

Exp.	Threads	Dur. (s)	Blocos	CPU (%)	Escritas (%)
1c·1	2×FF:1	1.23	1	100/0	100/0
1c·2	2×RR:1	1.90	12	62/38	51/49
1c·3	2×ID	1.59	236	50/50	50/50
1c·4	2×LI	1.53	226	51/49	51/49
1c·5	LI ID	1.55	220	31/69	33/67
1c·6	RR:10 FF:10	1.19	2	9/91	8/92
1c·7	RR:10 RR:20	1.45	1	0/100	0/100
1c·8	FF:1 ID	1.17	1	100/0	100/0
1c·9	OT ID	1.27	3	100/0	100/0
1c·10	OT LI	1.30	5	99/1	99/1
1c·11	2×ID 2×LI	1.79	369	35/34/16/15	33/34/16/16
2c·1	2×FF:1	4.77	757 mil	50/50	50/50
2c·2	2×RR:1	4.87	584 mil	50/50	49/51
2c·3	LI ID	5.04	497 mil	50/50	50/50
2c·4	4×RR:1	8.01	3176 mil	25/24/28/23	24/24/28/23
2c·5	2×ID 2×LI	5.71	3071 mil	29/29/22/21	28/29/22/21

1c/2c = 1 ou 2 núcleos. FF = FIFO, RR, OT = OTHER, ID = IDLE, LI = LOW_IDLE (FIFO/RR com a prioridade após ":"). CPU e escritas por thread na ordem A/B/C/D; blocos = total de escalonamentos.

3. O que os experimentos mostraram

Mesma política, 1 núcleo. FIFO (1c·1): a primeira thread a receber a CPU escreve 100% e só a larga ao terminar, pois não há fatia de tempo. RR (1c·2): as threads se revezam em fatias de ~100 ms (resumo ABABABABABAB). IDLE e LOW_IDLE (1c·3, 1c·4): o CFS divide ~50/50, trocando a cada ~4 ms (tick de 250 Hz) em ordem variável. Com pesos iguais, IDLE e LOW_IDLE são indistinguíveis.

Políticas diferentes, 1 núcleo. Tempo real × CFS: uma FIFO contra uma IDLE (1c·8) deixa a IDLE com 0% das escritas. RR × FIFO de mesma prioridade (1c·6, Fig. 3): A (RR) roda uma única fatia de ~97 ms, vai ao fim da fila e B (FIFO) não devolve mais a CPU (7,5% × 92,5%); a FIFO só é interrompida pelo RT throttling (950 ms por período de 1 s), quando a CPU fica ~50 ms com as tarefas normais (o main entra no pthread_join) e depois ociosa. Pesos no CFS: OTHER × IDLE (1c·9) dá 99,7/0,3% de CPU, exatamente a razão 1024:3; OTHER × LOW_IDLE (1c·10) deixa a LOW_IDLE com 0,6% (acima dos 0,1% do peso, provavelmente pelo crédito de vruntime ao acordar, relevante em execução tão curta).

LOW_IDLE × IDLE. No mesmo núcleo a IDLE recebe **69%** da CPU e a LOW_IDLE **31%** (1c·5, Fig. 1; o peso 1:3 prevê 75/25); com duas de cada (1c·11, Fig. 2), ~34,5% para cada IDLE e ~15,5% para cada LOW_IDLE (previsto 37,5/12,5). O trace mostra os dois mecanismos da política: (1) quando a IDLE acorda (sem_post da LOW_IDLE), a LOW_IDLE é preemptada no mesmo instante (estado R ~0,1 ms após a troca anterior); (2) a LOW_IDLE só recupera a CPU no tick, quando a IDLE esgota sua fatia, ou quando a IDLE dorme no semáforo. A distância da teoria vem do tick: a fatia ideal da LOW_IDLE (~1,5 ms) vira 4 ms na prática, enquanto a IDLE emenda até 3 ticks (12 ms).

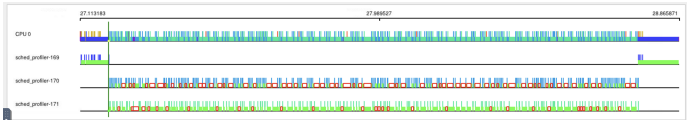
Prioridades. Com RR:10 × RR:20 (1c·7) a thread de prioridade 20 escreve 100% e a de prioridade 10 nunca recebe fatia: o revezamento do RR só ocorre entre threads de mesma prioridade, e a de maior prioridade sempre passa à frente. Entre tempo real e CFS a prioridade nem entra em jogo: qualquer FIFO/RR passa à frente de OTHER, IDLE e LOW_IDLE.

1 × 2 núcleos. Com uma CPU por thread a política deixa de importar: FIFO (2c·1, Fig. 5), RR (2c·2) e LOW_IDLE × IDLE (2c·3, Fig. 4) dividem ~50/50, e as threads mudam de CPU durante a execução. A diferença de peso só reaparece com mais threads que núcleos (2c·5: IDLE ~29%, LOW_IDLE ~21%). O paralelismo não ajuda este programa: só uma thread escreve por vez, então elas disputam o semáforo, dormem e acordam milhares de vezes (trocas voluntárias passam de dezenas para milhares) e a execução fica ~4× mais lenta (FIFO: 1,2 s × 4,8 s). O buffer passa a alternar quase letra a letra (centenas de milhares de blocos): com várias CPUs, contar blocos mede a disputa pelo semáforo, não o escalonador, e o tempo de CPU passa a ser a métrica confiável.

Conclusão. No mesmo núcleo, a política define quem roda: tempo real passa à frente de tudo, a prioridade decide entre tempo real, a fatia separa RR de FIFO, e no CFS a CPU segue o peso, com a SCHED_LOW_IDLE recebendo menos que a SCHED_IDLE, como projetado. Com núcleos suficientes para todas as threads, essas diferenças desaparecem.

4. KernelShark e saídas do programa

Em cada figura: linhas CPU (tarefa em execução) e uma linha por thread (verde = executando, vermelho = pronta, preto = dormindo); o primeiro PID é o main, os seguintes são A, B, C... Abaixo, a saída salva da mesma execução.



```
buffer = 5000000, threads = 2, cpus = 1
A: SCHED_LOW_IDLE prio 0
B: SCHED_IDLE prio 0
BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB...

Resumo:
BABABABABABABABABABABABABABABABABABABABABABABABABABABABABABABABABABA...

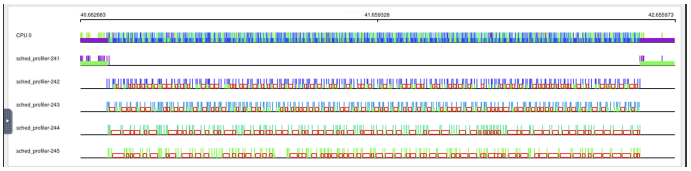
Escalonamentos:
A = 110
B = 110

Escritas no buffer:
A = 1651176 (33.0%)
B = 3348824 (67.0%)

Tempo de CPU:
A = 474.8 ms (30.9%)
B = 1063.3 ms (69.1%)

Trocas de contexto (voluntarias / involuntarias):
A = 58 / 178
B = 62 / 175
```

Fig. 1 – 1c·5, A = LOW_IDLE, B = IDLE (main = 169, A = 170, B = 171): B executa mais e por mais tempo a cada vez.



```
buffer = 5000000, threads = 4, cpus = 1
A: SCHED_IDLE prio 0
B: SCHED_IDLE prio 0
C: SCHED_LOW_IDLE prio 0
D: SCHED_LOW_IDLE prio 0
BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB...

Resumo:
BCDABABCADBADCBADCABADCBACDBABCDABCDACDBABADCBADCBABCDABCDAB...

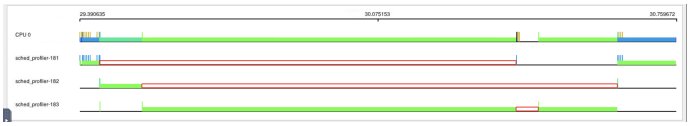
Escalonamentos:
A = 110
B = 107
C = 76
D = 76

Escritas no buffer:
A = 1655297 (33.1%)
B = 1714734 (34.3%)
C = 826902 (16.5%)
D = 803067 (16.1%)

Tempo de CPU:
A = 614.7 ms (34.7%)
B = 609.8 ms (34.4%)
C = 279.2 ms (15.8%)
D = 269.0 ms (15.2%)

Trocas de contexto (voluntarias / involuntarias):
A = 83 / 158
B = 82 / 151
C = 60 / 96
D = 55 / 105
```

Fig. 2 – 1c·11, A,B = IDLE, C,D = LOW_IDLE (main = 241, A–D = 242–245): as linhas de C e D têm menos verde.



```
buffer = 5000000, threads = 2, cpus = 1
A: SCHED_RR prio 10
B: SCHED_FIFO prio 10
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...

Resumo:
AB

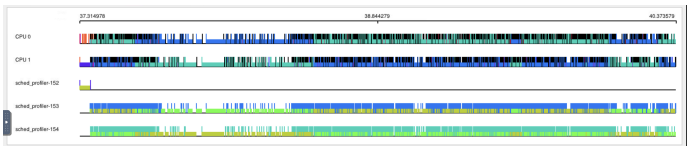
Escalonamentos:
A = 1
B = 1

Escritas no buffer:
A = 375999 (7.5%)
B = 4624001 (92.5%)

Tempo de CPU:
A = 97.7 ms (8.6%)
B = 1038.2 ms (91.4%)

Trocas de contexto (voluntarias / involuntarias):
A = 1 / 1
B = 1 / 1
```

Fig. 3 – 1c·6, A = RR:10, B = FIFO:10 (main = 181, A = 182, B = 183): uma fatia de A, B até o fim, pausa do RT throttling.



```
buffer = 5000000, threads = 2, cpus = 2
A: SCHED_LOW_IDLE prio 0
B: SCHED_IDLE prio 0
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...

Resumo:
ABABABABABABABABABABABABABABABABABABABABABABABABABABABABABABABABAB...

Escalonamentos:
A = 248516
B = 248515

Escritas no buffer:
A = 2474853 (49.5%)
B = 2525147 (50.5%)

Tempo de CPU:
A = 4539.8 ms (50.0%)
B = 4540.2 ms (50.0%)

Trocas de contexto (voluntarias / involuntarias):
A = 6454 / 65
B = 7418 / 54
```

Fig. 4 – 2c·3, A = LOW_IDLE, B = IDLE (main = 152, A = 153, B = 154), trecho de ~3 s: as duas executam o tempo todo.



```
buffer = 5000000, threads = 2, cpus = 2
A: SCHED_FIFO prio 1
B: SCHED_FIFO prio 1
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...

Resumo:
ABABABABABABABABABABABABABABABABABABABABABABABABABABABABABABABABAB...

Escalonamentos:
A = 378622
B = 378622

Escritas no buffer:
A = 2507653 (50.2%)
B = 2492347 (49.8%)

Tempo de CPU:
A = 4362.5 ms (49.6%)
B = 4438.3 ms (50.4%)

Trocas de contexto (voluntarias / involuntarias):
A = 5309 / 0
B = 3816 / 0
```

Fig. 5 – 2c·1, 2× FIFO:1 (main = 126, A = 127, B = 128): as duas em paralelo, em CPUs diferentes.